Scaffold Supplement — PricebookCalc Business Logic

This document provides the five missing business-logic sections that need to be pasted into the Google Workspace scaffold. Each section is self-contained.

---

1. System Calculator Formulas

Inputs

Input	Description
Target M&E %	Dropdown (15%–25% in 0.5% steps). Percentage of final price M&E should represent.
Materials Cost	Dollar amount of materials
Equipment Cost	Dollar amount of equipment
Labor Hours or Labor %	Depends on labor mode in Settings
Subcontractor	Additional cost added to price
Crane	Additional cost added to price
Permit	Additional cost added to price
Discount	Dollar amount or % of total to subtract
OEM Rebate	Added to revenue for GM% (not to price)
Utility Rebate	Added to revenue for GM% (not to price)

Outputs

Output	Description
displayPrice	Final selling price (optionally rounded up)
grossMarginPercent	(effectiveRevenue − totalAllCogs) / effectiveRevenue × 100
mePercent	totalMe / displayPrice × 100
estimatedLabor	laborCost + commissionCost + tax + financingCost (for ServiceTitan import)

Step-by-Step Calculation

```
Step 1: Tax & M&E
  salesTaxRate = parseFloat(settings.salesTaxPercent) / 100
  tax = (materials + equipment) × salesTaxRate
  totalMe = materials + equipment + tax

Step 2: Base Price from M&E Target
  targetMeRate = parseFloat(targetMe) / 100
  basePrice = targetMeRate > 0 ? totalMe / targetMeRate : 0
```

```
Step 3: Add Other COGS, Apply Discount
  totalOtherCogs = subcontractor + crane + permit
  priceBeforeDiscount = basePrice + totalOtherCogs

  If discountMode is "dollar":
    discountVal = discountInput
  If discountMode is "percent":
    discountVal = priceBeforeDiscount × (discountInput / 100)

  price = max(0, priceBeforeDiscount − discountVal)

Step 4: Commission (always % of job, with burden)
  commissionRate = parseFloat(settings.commissionPercent) / 100
  commissionBurdenRate = parseFloat(settings.commissionBurdenPercent) / 100
  commissionCost = (price × commissionRate) × (1 + commissionBurdenRate)

Step 5: Financing
  financingRate = parseFloat(settings.financingFeePercent) / 100
  financingCost = price × financingRate

Step 6: Labor Cost (depends on settings.laborMode)
  laborBurdenRate = parseFloat(settings.laborBurdenPercent) / 100
  If laborMode is "hours":
    hourlyRate = parseFloat(settings.laborHourlyRate)
    laborCost = (hours × hourlyRate) × (1 + laborBurdenRate)
  If laborMode is "percent":
    laborPercentRate = parseFloat(laborPercent) / 100
    laborCost = (price × laborPercentRate) × (1 + laborBurdenRate)

Step 7: Estimated Labor (for ServiceTitan)
  estimatedLabor = laborCost + commissionCost + tax + financingCost

Step 8: Display Price & Gross Margin
  displayPrice = roundUp ? Math.ceil(price) : price
  totalAllCogs = totalMe + commissionCost + financingCost + laborCost + totalOtherCogs
  effectiveRevenue = displayPrice + oemRebate + utilityRebate
  grossMarginAmount = effectiveRevenue − totalAllCogs
  grossMarginPercent = effectiveRevenue > 0
      ? (grossMarginAmount / effectiveRevenue) × 100 : 0
  mePercent = displayPrice > 0 ? (totalMe / displayPrice) × 100 : 0
```

---

2. Repair Calculator Formulas

Inputs

Input	Description
Parts Cost	Raw cost of parts
Material Markup	% markup on parts+tax (10%–300%)
Labor Rate	$/hour rate
Labor Hours	Number of hours
Burden %	Employer burden on labor/commission
Sales Tax %	Tax on parts
Commission %	% of total selling price (backed in)
Financing Fee %	% of total selling price
Highest Tech Pay Rate	For COGS calculation
Production Pay toggle	Switches between two labor modes
PSI Mode toggle	Preset values for PSI pricing

Mode Comparison

“$ + Hours” Mode (laborMode === “dollars”)

```
salesTax        = partsCost × taxRate
subtotal1       = partsCost + salesTax
markupAmount    = subtotal1 × markupRate
subtotal2       = subtotal1 + markupAmount

directLaborCost   = laborRate × laborHours
laborBurden       = directLaborCost × burdenRate
burdenedLaborCost = directLaborCost + laborBurden
subtotal3         = subtotal2 + burdenedLaborCost

denominator      = 1 − commissionRate × (1 + burdenRate) − financingRate
totalPriceCalc    = denominator > 0 ? subtotal3 / denominator : subtotal3

directCommission   = totalPriceCalc × commissionRate
commissionBurden   = directCommission × burdenRate
burdenedCommission = directCommission + commissionBurden
financingFee       = totalPriceCalc × financingRate

totalBeforeRounding = subtotal3 + burdenedCommission + financingFee
totalPrice       = roundUp ? Math.ceil(totalBeforeRounding) : totalBeforeRounding
```

“% of Job” Mode (laborMode === “percent”)

```
salesTax        = partsCost × taxRate
subtotal1       = partsCost + salesTax
markupAmount    = subtotal1 × markupRate
subtotal2       = subtotal1 + markupAmount

directLaborCost = 0          ← commission handles everything
subtotal3       = subtotal2  ← no labor added
```

```
denominator      = 1 − commissionRate × (1 + burdenRate) − financingRate
totalPriceCalc   = denominator > 0 ? subtotal3 / denominator : subtotal3

directCommission   = totalPriceCalc × commissionRate
commissionBurden   = directCommission × burdenRate
burdenedCommission = directCommission + commissionBurden
financingFee       = totalPriceCalc × financingRate

totalBeforeRounding = subtotal3 + burdenedCommission + financingFee
totalPrice       = roundUp ? Math.ceil(totalBeforeRounding) : totalBeforeRounding
```

COGS & Gross Margin

```
techLaborCost   = laborMode === "percent" ? 0 : highestTechRate × laborHours
burdenedTechLabor = techLaborCost × (1 + burdenRate)

COGS = partsCost + burdenedTechLabor + burdenedCommission
  (Financing is excluded from COGS — it's a pass-through fee)

grossMargin        = totalPrice − COGS
grossMarginPercent = totalPrice > 0 ? (grossMargin / totalPrice) × 100 : 0
```

Margin Health Thresholds

GM% Range	Label	Color
>= 50%	Strong	Green
>= 40%	Caution	Amber
< 40%	Low	Red

---

3. Reverse Calculator Formulas

The Reverse Calculator works backwards from a known selling price to determine the resulting gross margin percentage. It has two sub-modes: **System** and **Repair**.

System Sub-Mode

Inputs: Selling Price, Materials, Equipment, Labor (hours or %), Subcontractor, Crane, Permit, Discount ($ or %), OEM Rebate, Utility Rebate

```
tax          = (materials + equipment) × salesTaxRate
totalMe      = materials + equipment + tax
totalOtherCogs = subcontractor + crane + permit
```

```
Discount handling:
  If discountMode is "percent":
    discountVal = sellingPrice × discountInput / 100
  If discountMode is "dollar":
    discountVal = discountInput
  price = max(0, sellingPrice − discountVal)

commissionCost = (price × commissionRate) × (1 + commissionBurdenRate)
financingCost  = price × financingRate
laborCost      = (same logic as System Calculator, Step 6)
estimatedLabor = laborCost + commissionCost + tax + financingCost

totalAllCogs    = totalMe + commissionCost + financingCost + laborCost + totalOtherCog
effectiveRevenue = price + oemRebate + utilityRebate
grossMarginAmount  = effectiveRevenue − totalAllCogs
grossMarginPercent = effectiveRevenue > 0
    ? (grossMarginAmount / effectiveRevenue) × 100 : 0
mePercent = effectiveRevenue > 0 ? (totalMe / effectiveRevenue) × 100 : 0
```

Repair Sub-Mode

Inputs: Selling Price, Parts Cost, Material Markup, Labor Hours, Burden %, Sales Tax %, Commission %, Financing Fee %, Actual/Highest Tech Pay Rate, Production Pay toggle

Key difference: Uses user-entered selling price directly for commission/financing, rather than deriving price algebraically from costs.

```
salesTax      = partsCost × taxRate
subtotal1     = partsCost + salesTax            ← "Parts + Tax"
markupAmount  = subtotal1 × markupRate
materialsCost = subtotal1 + markupAmount        ← "Parts + Tax + Markup"

If laborMode is "dollars":
  directLaborCost   = highestTechRate × laborHours
  burdenedLaborCost = directLaborCost × (1 + burdenRate)
If laborMode is "percent":
  directLaborCost   = 0
  burdenedLaborCost = 0

commission = sellingPrice × commissionRate × (1 + burdenRate)
financing  = sellingPrice × financingRate

COGS = subtotal1 + burdenedLaborCost
  (Parts + Tax + direct labor; excludes markup and commission)

totalCosts         = COGS + financing
grossMargin        = sellingPrice − COGS
grossMarginPercent = sellingPrice > 0 ? (grossMargin / sellingPrice) × 100 : 0
mePercent        = sellingPrice > 0 ? (subtotal1 / sellingPrice) × 100 : 0
```

Price Comparison Chart (Repair Mode)

Two stacked vertical bars: **Calculated** (forward formula price) and **Selling** (user-entered price).

Forward "Calculated Price" formula:

```
denominator     = 1 − commissionRate × (1 + burdenRate) − financingRate
subtotal3       = materialsCost + burdenedLaborCost
calculatedPrice = subtotal3 / denominator
```

Stacked bar segments:

Segment	Color	Calculated Bar	Selling Bar
Parts + Tax	Gray	`subtotal1`	`subtotal1`
Markup	Blue	`markupAmount`	`markupAmount`
Labor	Green	`burdenedLaborCost`	`burdenedLaborCost`
Commission	Amber	`calculatedPrice × commissionRate × (1+burden)`	`sellingPrice × commissionRate × (1+burden)`
Financing	Violet	`calculatedPrice × financingRate`	`sellingPrice × financingRate`
Margin	Green/Red	~0 (by formula construction)	`sellingPrice − all above`

Margin segment is green when >= 0, red when negative. A static 3-column comparison table (Segment | Calculated | Selling) sits below the chart with a delta callout (`+$X above / −$X below formula price`).

---

4. Full Push Workflow Screens

Tab 1: Update Pricebook

Pushes calculated M&E pricing to existing services in ServiceTitan.

Input fields:

Field	Description
Tenant	Dropdown of configured ServiceTitan tenants
Tax Region	Location-based sales tax region selection
Categories	Multi-select from tenant's pricebook category tree
Target M&E %	Pricing target (15%–25% in 0.5% steps)
Sales Tax %	Tax rate applied to materials + equipment (auto-filled from region)
Labor Mode	Toggle: "Hours" or "% of Job"
Labor Hourly Rate	$/hour rate (shown when labor mode is "hours")

Field	Description
Labor %	% of job for labor (shown when labor mode is "percent")
Labor Burden %	Employer burden on labor
Commission %	% of selling price
Commission Burden %	Employer burden on commission
Financing Fee %	% of selling price
Round Up	Toggle to round price to next whole dollar

Price calculation per service:

```
materialsCost = Σ (material.cost × material.quantity) for linked materials
equipmentCost = Σ (equipment.cost × equipment.quantity) for linked equipment
rawMeCost     = materialsCost + equipmentCost
tax           = rawMeCost × salesTaxRate
totalMe       = rawMeCost + tax
newPrice      = totalMe / (targetMePercent / 100)
```

SSE progress flow (via `/api/servicetitan/:tenantId/push/progress`):

1. Client opens EventSource connection
2. Server emits progress events for phases: `services` → `materials` → `equipment` → `updating` → `complete`
3. Frontend displays real-time progress bars and status messages

Queue / concurrency limits:

Limit	Value	Notes
Max concurrent pushes	3	`MAX_CONCURRENT_PUSHES` in `pushQueue.ts`
Parallel updates per push	50	`CONCURRENCY` in `pushProcessor.ts`
Effective max API requests/sec	150	50 × 3 (batched; ServiceTitan rate limit is 60 req/sec, handled by retry/backoff)
Queue behavior	FIFO	`pushQueue` extends `EventEmitter`, emits `ready`

Exponential backoff retry (for 429 rate-limit errors):

```
For each retry attempt (max 3):
  delay = baseDelay × 2^attempt
  (1000ms, 2000ms, 4000ms)
Only retries on 429/rate-limit errors; other errors thrown immediately
```

Revert mechanism: Push history records store `originalValues` (JSON array of original prices). The revert endpoint restores these values via the same `updateService` API call.

Tab 2: New System Task

Creates a new system installation service in ServiceTitan.

Input fields:

Field	Required	Description
Code	Yes	Service code
Name (displayName)	Yes	Service display name
Description	No	Service description text
Warranty Description	No	Warranty text
Materials Cost	Yes	For M&E price calculation
Equipment Cost	Yes	For M&E price calculation
Target M&E %	Yes	Pricing target
Hours	No	Estimated labor hours
Taxable	No	Boolean checkbox
Allow Discount Codes	No	Boolean checkbox
Allow Membership Discounts	No	Boolean checkbox
GL Account	No	Searchable dropdown
Category	Yes	Category selection (validated before submit)
Image	No	File upload
PSI Mode toggle	No	Preset values for PSI pricing

Tab 3: New Repair Task

Creates a new repair service in ServiceTitan.

Input fields:

Field	Required	Description
Code	Yes	Service code
Name (displayName)	Yes	Service display name
Description	No	Service description text
Warranty Description	No	Warranty text
Parts Cost	Yes	Raw cost of parts
Material Markup	Yes	% markup on parts+tax
Labor Rate / Hours	Yes	For price calculation
Burden %	Yes	Employer burden
Sales Tax %	Yes	Tax on parts
Commission %	Yes	% of selling price
Financing Fee %	Yes	% of selling price
Hours	No	Estimated hours for ServiceTitan
Taxable	No	Boolean checkbox

Field	Required	Description
Allow Discount Codes	No	Boolean checkbox
Allow Membership Discounts	No	Boolean checkbox
GL Account	No	Searchable dropdown
Category	Yes	Category selection (validated before submit)
Image	No	File upload
Production Pay toggle	No	$ + Hours vs % of Job
PSI Mode toggle	No	Preset values for PSI pricing

API endpoints:

- `GET /api/servicetitan/:tenantId/gl-accounts` — Fetch GL accounts for dropdown
- `POST /api/servicetitan/:tenantId/services` — Create new service
- `POST /api/servicetitan/:tenantId/services/:serviceId/image` — Upload image

---

5. Payload Shaping

updateService — PATCH `/pricebook/v2/tenant/{tenantId}/services/{serviceId}`

```
{
  "price": 1234.56,
  "hours": 8.0
}
```

`price` and `hours` are the primary updatable fields. `memberPrice` is also accepted. The `cost` field is **read-only** — ServiceTitan calculates it internally from `hours × internal rate`.

> **Current push flow note**: The Update Pricebook processor (`pushProcessor.ts`) sends only `{ price }` per service. The `hours` field can be sent via the general update endpoint but is not set during bulk push operations.

createService — POST `/pricebook/v2/tenant/{tenantId}/services`

```
{
  "code": "SYS-001",
  "displayName": "2-Ton AC System Install",
  "description": "Full system installation including...",
  "warranty": {
    "duration": 12,
    "description": "One year parts and labor warranty"
  },
  "taxable": true,
  "active": true,
```

```
  "price": 8500.00,
  "memberPrice": 7650.00,
  "hours": 8.0,
  "categories": [12345],
  "account": "Sales",
  "allowDiscountCodes": true,
  "allowMembershipDiscounts": true
}
```

All fields except `code` and `displayName` are optional.

OAuth Token Exchange — POST `https://auth.servicetitan.io/connect/token`

```
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id={clientId}
&client_secret={clientSecret}
```

Response:

```
{
  "access_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

Tokens are cached in-memory per tenant until `expires_in − 60` seconds (for a typical 3600s token, cache lasts ~59 minutes).

ServiceTitan Field Mapping

PricebookCalc Field	ServiceTitan API Property	Updatable?	Notes
Selling Price	`price`	Yes	Static price sent via PATCH
Member Price	`memberPrice`	Yes	Optional discounted price
Estimated Labor Cost	`cost`	**No**	Calculated by ST from Hours × internal rate

PricebookCalc Field	ServiceTitan API Property	Updatable?	Notes
Hours	`hours`	Yes	Updatable via both create and update

Authenticated Request Headers

All API calls after token exchange include:

```
Authorization: Bearer {access_token}
ST-App-Key: {appKey}
Content-Type: application/json
```